DBMS

Interview Preparation Guide

Top 20 Most Asked Questions & Answers
Intermediate to Advanced Level | With Easy Explanations

Level	Count	Topics Covered
Intermediate	12	DBMS vs File System, Keys, Normalization, Normal Forms, SQL Joins, ACID, Indexing, Aggregates, DDL/DML/DCL/TCL, Views, ER Model
Advanced	8	Isolation Levels, Deadlocks, Clustered vs Non-Clustered Index, SQL vs NoSQL, Sharding, Query Optimization, Triggers vs Procedures, 2PL

Structured for quick revision before technical interviews

Table of Contents

Q1.	What is DBMS, and how is it different from a File System?	Intermediate
Q2.	What are Primary Key, Foreign Key, and Candidate Key?	Intermediate
Q3.	What is Normalization, and why is it important?	Intermediate
Q4.	Explain 1NF, 2NF, and 3NF with simple examples.	Intermediate
Q5.	What are the different types of SQL Joins?	Intermediate
Q6.	What are the ACID properties of a transaction?	Intermediate
Q7.	What is Indexing in a database, and why does it improve performance?	Intermediate
Q8.	What is the difference between Primary Key and Unique Key?	Intermediate
Q9.	What are Aggregate Functions, and how does GROUP BY work with them?	Intermediate
Q10.	What are DDL, DML, DCL, and TCL in SQL?	Intermediate
Q11.	What is a View in SQL?	Intermediate
Q12.	What is the ER Model, and what are Entities, Attributes, and Relationships?	Intermediate
Q13.	Explain the SQL Isolation Levels and the concurrency problems they prevent.	Advanced
Q14.	What is a Deadlock in a database context, and how is it typically resolved?	Advanced
Q15.	What is the difference between a Clustered Index and a Non-Clustered Index?	Advanced
Q16.	What is the difference between SQL and NoSQL databases?	Advanced
Q17.	What is Sharding, and how is it different from Partitioning?	Advanced
Q18.	How does a database optimize and execute a SQL query?	Advanced
Q19.	What is the difference between a Trigger and a Stored Procedure?	Advanced
Q20.	What is Two-Phase Locking (2PL) and how does it ensure serializability?	Advanced

Q1 **DBMS Basics** Intermediate

What is DBMS, and how is it different from a File System?

ANSWER

A DBMS (Database Management System) is software that lets you create, store, retrieve, update, and manage data in an organized way, while enforcing rules like data integrity, security, and concurrent access control.

A plain File System just stores raw files with no built-in structure or relationships between data — no concept of queries, no automatic consistency checks, no concurrency control, and duplicate/inconsistent data is common. A DBMS solves all of these problems using structured tables, constraints, and a query language (SQL).

EASY EXPLANATION: A file system is like keeping loose paper receipts stuffed in random drawers — you can find things, but slowly and with duplicates and mistakes. A DBMS is like a well-organized accounting ledger with rules: every entry is validated, cross-checked, and instantly searchable.

Q2 **Keys in DBMS** Intermediate

What are Primary Key, Foreign Key, and Candidate Key?

ANSWER

A Candidate Key is any column (or combination of columns) that can uniquely identify a row in a table — there can be multiple candidate keys in a table.

A Primary Key is the candidate key chosen to be THE main unique identifier for a table. It cannot contain NULL values and there can only be one primary key per table.

A Foreign Key is a column in one table that references the Primary Key of another table, used to establish and enforce a link/relationship between the two tables.

EASY EXPLANATION: In a classroom, Roll Number and Aadhar Number could both uniquely identify a student (candidate keys), but the school picks Roll Number as the official ID (primary key). A Foreign Key is like writing a student's Roll Number on their library card — it links the card back to exactly one student in the student table.

Q3 **Normalization** Intermediate

What is Normalization, and why is it important?

ANSWER

Normalization is the process of organizing data in a database to reduce redundancy (duplicate data) and avoid update, insertion, and deletion anomalies, by breaking large tables into smaller, related tables following a series of rules called 'normal forms' (1NF, 2NF, 3NF, BCNF, etc.).

It's important because it keeps data consistent (updating one place doesn't leave stale copies elsewhere), saves storage space, and makes the database easier to maintain.

EASY EXPLANATION: Imagine a single spreadsheet where every order row repeats the full customer address. If a customer moves, you'd have to update dozens of rows — and might miss some, causing

inconsistent data. Normalization splits this into a Customers table and an Orders table, so the address is stored just once and always stays correct.

Q4 **Normal Forms (1NF, 2NF, 3NF)** Intermediate

Explain 1NF, 2NF, and 3NF with simple examples.

ANSWER

1NF (First Normal Form): every column must hold only atomic (single, indivisible) values — no repeating groups or lists inside a single cell. Example: a 'Phone Numbers' column shouldn't store '9876543210, 9123456789' together in one field.

2NF (Second Normal Form): must already be in 1NF, AND every non-key column must depend on the ENTIRE primary key (this only matters when you have a composite/multi-column primary key) — no 'partial dependency'.

3NF (Third Normal Form): must already be in 2NF, AND there should be no 'transitive dependency' — a non-key column should not depend on another non-key column, only directly on the primary key.

EASY EXPLANATION: 1NF = one value per box, no comma-separated lists jammed into a cell. 2NF = every fact in a row must depend on the FULL ID, not just part of it (e.g., in an Order+Product table, 'Product Name' depends only on Product ID, not Order ID, so it shouldn't be stuck there). 3NF = don't let one plain fact depend on another plain fact instead of the actual ID — e.g., 'City' shouldn't depend on 'Zip Code' inside a Customer table; it should trace back to the Customer ID directly, or go into its own table.

Q5 **SQL Joins** Intermediate

What are the different types of SQL Joins?

ANSWER

INNER JOIN - returns only the rows that have matching values in both tables.

LEFT (OUTER) JOIN - returns all rows from the left table, plus matched rows from the right table (NULLs for no match).

RIGHT (OUTER) JOIN - returns all rows from the right table, plus matched rows from the left table (NULLs for no match).

FULL (OUTER) JOIN - returns all rows from both tables, with NULLs where there's no match on either side.

SELF JOIN - a table is joined with itself, useful for comparing rows within the same table (e.g., finding employees and their managers in the same Employees table).

EASY EXPLANATION: Imagine two guest lists — Table A (invited) and Table B (attended). INNER JOIN = people who were both invited AND attended. LEFT JOIN = everyone invited, showing who among them attended (blank if not). FULL JOIN = everyone from both lists combined, showing the full picture including no-shows and gate-crashers.

Q6 **ACID Properties** Intermediate

What are the ACID properties of a transaction?

ANSWER

Atomicity - a transaction is treated as a single indivisible unit; either ALL of its operations succeed, or NONE of them do (all-or-nothing).

Consistency - a transaction must bring the database from one valid state to another, respecting all defined rules, constraints, and relationships.

Isolation - concurrent transactions should not interfere with each other; the result should be as if transactions ran one after another (serially), even if they actually ran in parallel.

Durability - once a transaction is committed, its changes are permanent and will survive even a system crash or power failure.

EASY EXPLANATION: Think of transferring money between two bank accounts: Atomicity = money is either fully deducted AND fully added, never stuck halfway. Consistency = the total money in the bank stays mathematically correct before and after. Isolation = if two transfers happen at once, they don't corrupt each other's numbers. Durability = once you get the 'Transfer Successful' message, that change is permanent even if the bank's server crashes one second later.

Q7 **Indexing** Intermediate

What is Indexing in a database, and why does it improve performance?

ANSWER

An index is a special data structure (typically a B-Tree or B+ Tree) that stores a sorted reference to table data, allowing the database to find rows much faster than scanning the entire table row-by-row (called a 'full table scan').

Indexes dramatically speed up SELECT queries and searches (especially with WHERE, JOIN, and ORDER BY clauses), but they add overhead to INSERT, UPDATE, and DELETE operations since the index itself must also be updated whenever data changes.

EASY EXPLANATION: An index is like the index page at the back of a textbook. Without it, to find 'Photosynthesis' you'd flip through every single page (full table scan). With the index, you jump straight to 'Page 142' instantly. But every time new content is added to the book, someone also has to go update that index page too — that's the trade-off.

Q8 **Primary Key vs Unique Key** Intermediate

What is the difference between Primary Key and Unique Key?

ANSWER

A Primary Key uniquely identifies each row in a table, cannot contain NULL values, and a table can have only ONE primary key (though it can be composite, spanning multiple columns).

A Unique Key also ensures all values in a column are distinct, but it CAN allow one NULL value (in most databases), and a table can have MULTIPLE unique keys.

EASY EXPLANATION: The Primary Key is like a person's official government ID number — there's exactly one, and everyone must have it. A Unique Key is like an email address — most people have one

and no two people share the same one, but it's technically okay for someone to not have one on file (NULL), and you could track multiple such unique fields (email, phone number) at once.

Q9 **Aggregate Functions & GROUP BY** Intermediate

What are Aggregate Functions, and how does GROUP BY work with them?

ANSWER

Aggregate functions perform a calculation on a SET of rows and return a single summarized value. Common ones: COUNT() (number of rows), SUM() (total), AVG() (average), MIN() (smallest value), MAX() (largest value).

GROUP BY splits the result set into groups based on one or more columns, and the aggregate function is then applied SEPARATELY to each group rather than the whole table — for example, getting the total sales PER region instead of one grand total.

EASY EXPLANATION: Imagine a class of students grouped by section (Section A, B, C). Without GROUP BY, AVG(marks) gives one single average for the entire class. With 'GROUP BY Section', you instead get a separate average for Section A, a separate one for Section B, and a separate one for Section C.

Q10 **DDL, DML, DCL, TCL** Intermediate

What are DDL, DML, DCL, and TCL in SQL?

ANSWER

DDL (Data Definition Language) - defines/modifies database structure: CREATE, ALTER, DROP, TRUNCATE.

DML (Data Manipulation Language) - manipulates the actual data: SELECT, INSERT, UPDATE, DELETE.

DCL (Data Control Language) - controls access/permissions: GRANT, REVOKE.

TCL (Transaction Control Language) - manages transactions: COMMIT, ROLLBACK, SAVEPOINT.

EASY EXPLANATION: Think of building and running a shop: DDL is like building the shelves and layout (structure). DML is like stocking, selling, and removing items (data). DCL is like deciding who has keys to the storeroom (permissions). TCL is like finalizing a sale (COMMIT) or cancelling it and putting items back (ROLLBACK) before it's final.

Q11 **Views** Intermediate

What is a View in SQL?

ANSWER

A View is a virtual table based on the result of a stored SQL query. It doesn't physically store data itself (in most cases) — instead, it dynamically pulls data from the underlying base tables every time it's queried.

Views are used to simplify complex queries (hide complicated JOINs behind a simple SELECT * FROM view_name), restrict access to specific columns/rows for security, and present a customized version of data without duplicating it.

EASY EXPLANATION: A view is like a customized report generated fresh from the master ledger every time you ask for it — it shows you exactly the columns and rows you're allowed to see, without actually creating a separate physical copy of the underlying data.

Q12 **Entity-Relationship (ER) Model** Intermediate

What is the ER Model, and what are Entities, Attributes, and Relationships?

ANSWER

The ER (Entity-Relationship) Model is a conceptual, diagram-based way of designing a database before it's actually built, representing real-world things and how they connect to each other.

An Entity is a real-world object or concept (e.g., Student, Course). An Attribute is a property/detail of an entity (e.g., Student Name, Student Age). A Relationship describes how two or more entities are connected (e.g., a Student 'Enrolls In' a Course).

EASY EXPLANATION: An ER diagram is like a blueprint drawn before constructing a building. Entities are like the rooms (Student, Course), attributes are the labeled details on each room (size, purpose), and relationships are the doors and hallways connecting rooms together — all planned out before actual construction (building the real database) begins.

Advanced Level Questions

Q13 **Isolation Levels & Concurrency Issues** Advanced

Explain the SQL Isolation Levels and the concurrency problems they prevent.

ANSWER

There are 4 standard isolation levels, from weakest to strongest: Read Uncommitted, Read Committed, Repeatable Read, and Serializable — each trades off performance for stricter data consistency.

Dirty Read - reading uncommitted (possibly to-be-rolled-back) data from another transaction; prevented by Read Committed and above.

Non-Repeatable Read - re-reading the same row within a transaction gives a different value because another transaction modified and committed it in between; prevented by Repeatable Read and above.

Phantom Read - re-running the same query within a transaction returns a different SET of rows because another transaction inserted/deleted matching rows; prevented only by Serializable, the strictest level.

EASY EXPLANATION: Dirty Read is like reading someone's rough draft before they've even saved it — the draft might get deleted entirely (rolled back), making your reading pointless. Non-Repeatable Read is like checking a stock price twice in one report and getting two different numbers because someone updated it in between your two checks. Phantom Read is like counting 'how many people are in the room' twice, and getting a different headcount the second time because someone walked in or out between your two counts.

Q14 **Deadlock in DBMS** Advanced

What is a Deadlock in a database context, and how is it typically resolved?

ANSWER

A deadlock occurs when two or more transactions are each waiting for a lock held by the other, so none of them can proceed — forming a circular waiting chain.

Most databases handle this using automatic Deadlock Detection: the DBMS periodically builds a 'wait-for graph' of which transaction is waiting on which, and if it finds a cycle, it picks one transaction as the 'victim', automatically rolls it back, and releases its locks so the others can continue.

EASY EXPLANATION: Two transactions are like two people trying to pass each other in a narrow hallway from opposite sides, each waiting for the other to step aside first — neither moves, forever. The database resolves this the way a referee would: it picks one person, gently pushes them backward (rolls back that transaction) so the other can pass through.

Q15 **Clustered vs Non-Clustered Index** Advanced

What is the difference between a Clustered Index and a Non-Clustered Index?

ANSWER

A Clustered Index determines the PHYSICAL storage order of data rows in the table itself — since data can only be physically sorted one way, a table can have only ONE clustered index (usually built

automatically on the primary key).

A Non-Clustered Index is a SEPARATE structure that stores pointers (references) back to the actual data rows, without changing the physical order of the table itself. A table can have MULTIPLE non-clustered indexes, but lookups require an extra step of following the pointer back to the actual row.

EASY EXPLANATION: A Clustered Index is like a dictionary where the actual word definitions are physically printed in alphabetical order on the pages — there's only one possible physical order. A Non-Clustered Index is like the separate index at the back of a different book that just tells you 'Photosynthesis -> Page 142' — the book's actual pages aren't reordered, and you can have several such indexes (by topic, by author, etc.) pointing back to the same unordered pages.

Q16 **SQL vs NoSQL** Advanced

What is the difference between SQL and NoSQL databases?

ANSWER

SQL (Relational) databases store data in structured tables with fixed schemas (predefined columns and data types), use SQL as the query language, strongly support ACID transactions, and scale primarily VERTICALLY (adding more power to a single server). Examples: MySQL, PostgreSQL, Oracle.

NoSQL (Non-relational) databases use flexible/dynamic schemas and various data models — document-based (MongoDB), key-value (Redis), column-family (Cassandra), or graph-based (Neo4j). They typically prioritize scalability and flexibility over strict ACID guarantees (often following 'eventual consistency' instead), and scale primarily HORIZONTALLY (adding more servers).

EASY EXPLANATION: SQL databases are like a strict, pre-printed form where every field is fixed in advance — great for consistency but rigid. NoSQL databases are like a flexible notebook where each entry can have a different structure depending on what's needed — great for rapidly changing or varied data, but with looser rules about strict consistency.

Q17 **Database Sharding & Partitioning** Advanced

What is Sharding, and how is it different from Partitioning?

ANSWER

Partitioning is dividing a single large table into smaller pieces (partitions) WITHIN the same database server, based on some column value (e.g., partitioning an Orders table by year). It helps manage large tables more efficiently but all pieces still live on one machine.

Sharding is a specific type of horizontal partitioning where the pieces (shards) are distributed ACROSS MULTIPLE separate database servers/machines. Each shard holds a subset of the total data, allowing the database to scale beyond what a single server could handle, at the cost of added complexity (like handling cross-shard queries and joins).

EASY EXPLANATION: Partitioning is like splitting one giant filing cabinet into labeled drawers within the same room. Sharding is like splitting those files across filing cabinets in entirely different buildings/branches — each branch handles its own subset of files independently, which lets you scale far beyond what one room/server could ever hold, but finding a file that spans multiple branches becomes more complex.

Q18 **Query Optimization & Execution Plan** Advanced

How does a database optimize and execute a SQL query?

ANSWER

When a query is submitted, the database's Query Optimizer analyzes multiple possible ways ('execution plans') to run it — such as which indexes to use, the order to join tables in, and whether to scan the full table or use an index — and estimates the cost (in terms of I/O, CPU) of each plan using statistics about the data (like table size and value distribution).

It then picks the plan with the lowest estimated cost and executes it. Developers can inspect this using commands like EXPLAIN or EXPLAIN ANALYZE, which reveal exactly how the database plans to fetch the data — useful for identifying slow queries, like ones performing unnecessary full table scans instead of using an available index.

EASY EXPLANATION: It's like a GPS navigation app calculating multiple possible routes to a destination (via highway, via local roads, via a shortcut) and picking the one it estimates will take the least time, based on real-time traffic data (statistics). You can also 'peek' at the chosen route (EXPLAIN) to understand why the GPS picked it, and spot if it accidentally chose a slow, unnecessary detour.

Q19 **Trigger vs Stored Procedure** Advanced

What is the difference between a Trigger and a Stored Procedure?

ANSWER

A Stored Procedure is a precompiled, reusable block of SQL code that must be EXPLICITLY called/invoked by a user, application, or another program whenever it needs to run (e.g., CALL calculate_bonus()).

A Trigger is a block of SQL code that runs AUTOMATICALLY in response to a specific event on a table — such as BEFORE or AFTER an INSERT, UPDATE, or DELETE — without anyone explicitly calling it. It's commonly used for enforcing business rules, auditing/logging changes, or maintaining derived/summary data automatically.

EASY EXPLANATION: A Stored Procedure is like a recipe card that a chef pulls out and follows only when they decide to cook that specific dish. A Trigger is like an automatic kitchen alarm that goes off BY ITSELF the moment a certain event happens — say, whenever the oven door opens — without any chef needing to manually trigger it.

Q20 **Two-Phase Locking (2PL)** Advanced

What is Two-Phase Locking (2PL) and how does it ensure serializability?

ANSWER

Two-Phase Locking is a concurrency control protocol that ensures transactions execute in a way equivalent to some serial (one-at-a-time) order, by dividing each transaction's locking behavior into two distinct phases.

Growing Phase - the transaction can ACQUIRE locks (shared or exclusive) as needed, but cannot release any lock yet.

Shrinking Phase - once the transaction releases even a single lock, it enters this phase and can no longer acquire any NEW locks, only release the ones it already holds. This strict separation guarantees serializability, though basic 2PL can still be vulnerable to deadlocks (which is why 'Strict 2PL', holding all locks until commit/rollback, is commonly used in practice).

EASY EXPLANATION: Imagine a strict rule at a buffet: you can keep taking new plates of food (acquiring locks) ONLY as long as you haven't returned any plate yet. The moment you return your very first plate (release a lock), you're not allowed to grab any new plates anymore — you can only keep returning the ones you already have. This strict 'grab everything first, return everything after' discipline prevents messy overlaps between different people's (transactions') buffet rounds.